🏭 Microservices Inventory Management System

Event-Driven • Fault-Tolerant • Scalable Retail Backend

LogicVeda Web Development Domain Capstone Project
March 2026

Title: Microservices-Based Inventory Management System
subtitle: Distributed, Resilient E-Commerce Backend Platform
author: LogicVeda
prepared-for: LogicVeda – Web Development Domain
date: March 2026
Project Code: lv1-2026-03-02
version: 2.0 – Detailed Professional Edition

📌 Executive Summary A production-grade microservices-based inventory & order management system that demonstrates service decomposition, event-driven communication, resilience patterns, API gateway usage, observability, and Kubernetes-ready deployment — mirroring architectures used at scale by Amazon, Shopify, etc.

1. Project Overview & Business Value

Objective

Build a modular, independently deployable backend for e-commerce/retail inventory operations, handling product catalog, stock management, orders, and reporting with high availability and fault tolerance.

Target Use Cases

- E-commerce startups
- Retail chains with multiple warehouses
- Supply chain SaaS platforms

Core Business Value

- Independent scaling of services (e.g., Orders vs Catalog)
- Eventual consistency for stock updates
- Zero-downtime deployments
- Deep DevOps & distributed systems mastery

Non-Functional Requirements

- Throughput: ≥ 1,000 req/min per service
- Resilience: Circuit breaking, retries, timeouts
- Observability: Distributed tracing + logs + metrics
- Data consistency: Eventual via sagas / 2PC alternatives

2. Detailed Functional Requirements

ID	Feature	Description	Acceptance Criteria
F-01	Service Discovery & API Gateway	Kong / custom gateway routing + auth proxy	All requests routed correctly; JWT validation at gateway
F-02	Authentication Service	JWT issuance, validation, role management	Register/login, token validation across services
F-03	Inventory Service	Product CRUD, stock levels, low-stock alerts	Stock updates atomic; optimistic locking
F-04	Orders Service	Order creation, status tracking, payment simulation	Saga pattern for order → stock deduction → confirmation

ID	Feature	Description	Acceptance Criteria
F-05	Event Bus	Async communication (order-placed → stock-update)	At-least-once delivery; idempotency
F-06	Reporting & Search	Elasticsearch-powered product search + admin dashboards	Aggregations, faceted search, real-time stock metrics
F-07	Resilience Patterns	Circuit breakers, retries, timeouts, bulkheads	Graceful degradation during partial failures
F-08	Observability	Structured logging, metrics, distributed tracing	Request correlation ID across services

3. Technology Stack – 2026 Production Grade

Category	Selection	Rationale / Alternatives Considered
Services Framework	Node.js 22 • Express / NestJS	NestJS for better structure in larger systems
API Gateway	Kong / Traefik / Express Gateway	Kong chosen for plugin ecosystem
Messaging	Kafka / RabbitMQ	Kafka for durability & ordering
Database	PostgreSQL (relational) + Elasticsearch	ACID for orders + search/analytics
Cache	Redis	Session + rate limiting
Containerization	Docker	Multi-stage builds
Orchestration	Kubernetes (Deployments, Services, HPA)	Helm charts optional
CI/CD	GitHub Actions	Matrix strategy for multi-service testing
Observability	ELK Stack / Prometheus + Grafana	Jaeger for tracing
Resilience	Polly-like patterns (Node) or built-in retries	Hystrix-style circuit breakers

4. Extremely Granular 28-Day Execution Plan

Week 1 – Architecture, Auth & Core Services

Day 1

• Domain-driven design session • bounded contexts

• Service map + OpenAPI specs

Day 2

• Monorepo setup (turborepo / nx)

• Auth service boilerplate + JWT

Day 3

• PostgreSQL schema + migrations (Prisma / TypeORM)

• Inventory service CRUD

Day 4

• API Gateway setup + routing rules

• JWT validation plugin/middleware

Day 5

• Docker Compose for local dev (all services + DB + Redis)

Day 6

• Basic inter-service call (Inventory → Auth)

Day 7

• Week 1 tag • Postman collection • local smoke test

Week 2 – Event-Driven Core & Orders

Day 8–9

• Kafka / RabbitMQ setup + topics (order-placed, stock-updated)

Day 10

• Orders service • order creation flow

Day 11

• Saga implementation (compensating transactions)

Day 12

• Event consumers in Inventory & Orders

Day 13

• React admin dashboard bootstrap + auth

Day 14

• End-to-end order placement test

Week 3 – Resilience, Search & Analytics

Day 15

• Circuit breakers + retries (axios-retry / custom)

Day 16

• Elasticsearch integration • product indexing

Day 17

• Search endpoint + faceted filters

Day 18

• Reporting service • Chart.js dashboards

Day 19

• ELK logging setup + correlation IDs

Day 20

• Chaos testing (kill service, network delay)

Day 21

• Week 3 tag • resilience report

Week 4 – Orchestration & Production

Day 22

• Kubernetes manifests for all services

Day 23

• Ingress + HPA configuration

Day 24

• GitHub Actions multi-service pipeline

Day 25

• Deploy to cluster (EKS / GKE / DigitalOcean)

Day 26

• Grafana dashboards + alerts

Day 27

• Demo video recording

Day 28

• Final QA • README • PDF export

LogicVeda – Web Development Domain

Project Submission Guidelines

March 2026 Edition

Prepared for participants

Focus: Industry-grade full-stack / advanced web projects

Submission period: You need to submit the project on or before the due date if you submit after the due date means you will be automatically disqualified for stipend by the system.

Evaluation emphasis: Code quality · Documentation · Live demo · Scalability & security awareness

1. General Rules & Eligibility

- All code must be **original work** created mainly during the LogicVeda preparation/submission window.
- Participants must submit **exactly 1 project** (as per the planned structure: 1 month each, weekly breakdown).
- Projects should demonstrate **progressive complexity**:
    1. Strong modern frontend / real-time foundations
    2. Distributed systems / microservices / resilience
    3. AI/ML-integrated or data-heavy web application
- Plagiarism / Ai generated content will result in disqualification for stipend.

2. Mandatory Submission Deliverables

Submit **one consolidated package** containing all three projects:

#	Deliverable	Format / Location	Required?	Evaluation Weight
1	Project Documentation / Report (1 PDF)	1 PDF file (A4, 8–15 pages)	Yes	25%
2	Live Public Demo URL	1 HTTPS link (no mandatory login)	Yes	30%
3	GitHub Repository	1 repo with clear folders	Yes	20%

#	Deliverable	Format / Location	Required?	Evaluation Weight
4	README.md per project	Detailed, professional README in repo	Yes	15%
5	Demo Video(s)	3–7 min per project (YouTube unlisted / Loom / Drive)	Yes	10%

Naming convention recommendation

`YourName_Project1_RealTimeCollab_LogicVeda_March2026.pdf/zip`

3. Documentation Guidelines – What every PDF should contain

Use consistent structure across all three documents for a polished portfolio feel.

1. **Hero / Cover Section**
    - Large project title
    - Catchy tagline
    - Your name + @harsadash + date
    - Gradient background (optional but recommended)
2. **Project Overview**
    - Vision & objectives
    - Target users / use cases
    - Business value delivered
    - Non-functional goals (latency, concurrency, availability targets)
3. **Key Features** (table format)
   | ID | Feature | Description | Acceptance Criteria |
4. **Technology Stack** (table)
   | Category | Technology | Rationale / Alternatives |
5. **Architecture Diagram**
    - Include screenshot (Excalidraw, Draw.io, Lucidchart, etc.)
    - Or clean ASCII art if no image tool used
6. **Detailed Execution Timeline**

    - Day-by-day or week-by-week breakdown
    - Major deliverables per phase
    - Milestones & checkpoints

7. **Technical Highlights**
    - Security measures (OWASP mitigation, input sanitization, rate limiting, etc.)
    - Performance & scalability notes (caching, lazy loading, load test results)
    - Challenges faced & how solved

8. **Deployment & Operations**
    - Platform used (Vercel, Render, Railway, AWS, Fly.io, etc.)
    - CI/CD pipeline summary
    - Monitoring / health checks (if implemented)

9. **Visuals**
    - 5–10 high-quality screenshots / GIFs
    - Architecture diagram
    - Live demo captures

10. **Personal Reflection** (optional but strongly recommended)
    - Key learnings
    - Industry best practices applied
    - Future roadmap ideas

4. Code & Repository Expectations

- Clean, modular code structure
- Consistent naming & formatting (use ESLint / Prettier)
- Semantic commit messages (`feat: add real-time cursor presence`, `fix: JWT refresh token bug`, etc.)
- Feature branches & pull requests (even if solo)
- `.gitignore` properly configured
- No committed secrets / API keys
- Basic tests appreciated (even 30–50% coverage shows intent)
- Responsive design + accessibility basics (ARIA labels, keyboard navigation)

5. Live Demo Requirements

- Publicly accessible (no VPN / geo-restriction)
- HTTPS enforced
- Core functionality available without sign-up (demo account OK if needed)
- Fast initial load (< 5 seconds preferred)
- Mobile responsive (test on phone / tablet)
- Include brief instructions on demo page if non-obvious

6. Evaluation Criteria (Suggested 100-point scale)

Category	Points	Focus Areas
Innovation & Problem Solving	15	Original features, thoughtful design choices
Technical Depth & Best Practices	25	Clean code, modern stack, security & performance awareness
Functionality & User Experience	20	Works as promised, intuitive UX, responsive
Documentation Quality	20	Clear, professional, well-structured
Deployment & Reliability	10	Stable live demo, easy local setup
Presentation & Polish	10	Demo video quality, visual appeal of docs

7. Important Rules & Tips

- **Strict deadline** – no late submissions accepted
- **File size limits**: ZIP ≤ 500 MB, individual PDFs ≤ 10 MB
- **No external dependencies that require payment / private keys** for judges to run demo
- **Backup plan**: If live demo is down, high-quality video + screenshots must still convey full functionality
- **Stand-out practices**:
  - Include Lighthouse scores / performance metrics
  - Show real multi-user session in video (especially Project 1)
  - Mention OWASP mitigations, load test results, CI/CD pipeline
  - End docs with personal growth reflection

Submission email / form subject example

```
Thers no email submission you need to submit the project in dashboard itself
```

Good luck!
Submit with confidence — your detailed planning and advanced topics already make these projects stand out.

Team LogicVeda
March 2026